أكاديمية كاوست
KAUST ACADEMY

جامعة الملك عبدالله للعلوم والتقنية
King Abdullah University of Science and Technology

Batch Normalization using an SVD Layer: From Regression to the Official RRAE Autoencoder Setting

Abdulmohsen Mohammedsaleh, Majed Alsulami, Yousef Alogiley, Jad Mounayer

abdulmohsen1284@gmail.com, majedsaadalsulami@gmail.com, yousef.alogiley@gmail.com, Jad.mounayer@outlook.com

Abstract

In Week 3 we returned to the source: we re-read our mentor's official **RR_layer** repository line by line and catalogued every difference between its example pipeline and the study we built in Weeks 1–2. The comparison revealed that the official example is not a regression task at all — it is an **autoencoder** whose Rank-Reduction (RR) layer sits in the *latent space* (dimension 200, rank 2, matching the data's two intrinsic degrees of freedom), trained on *grid-placed* Gaussian centers, and judged by *latent-space quality* (interpolation and random generation) rather than by prediction error. We therefore reproduced the official autoencoder experiment under our Report-2 protocol (seeded data, multiple seeds, mean ± std, from-scratch BatchNorm), added a BN variant to keep our project's central comparison, and introduced *quantitative* latent-space metrics on top of the repository's visual checks. Across 15 runs, the outcome is sharply two-sided. On the probe the repository highlights — random generation from the latent space — the RRAE beats the baseline autoencoder by 7.9× on every seed under both data layouts, the project's first clear win for the SVD layer; BN helps neither probe. The price is reconstruction: 61× the baseline's error at a 40-epoch budget. Our inference-basis ablation *exonerates* the repository's basis construction — bank, full-data, and even per-batch-oracle bases agree to within 1.4° and to the third decimal in error — so the cost is the rank-2 information floor itself, and our fork's `finalize_basis_from_data` addition turns out to be unnecessary. A 200-epoch check (the official budget) shrinks the RRAE's reconstruction gap 8.3×, makes its interpolation comparable to the baseline's, and grows its generation advantage to 37× — our 40-epoch tables understate the layer. We also document three concrete code-level findings between the repository and the copy used in our notebooks.

Date: August 8, 2026

Code: https://github.com/Yousef10p/RR-Layer

Contents

1 Introduction

In Weeks 1–2 we compared three otherwise-identical convolutional networks — a plain baseline (OA), a Batch-Normalization variant (OA_BN) [1], and a Rank-Reduction variant (OA_RR) built on the RR layer of Rank Reduction Autoencoders [2] — on a supervised Gaussian-blob *center-regression* task. The multi-seed study of Report 2 confirmed a consistent ranking on that task: the plain baseline wins, RR sits in the middle, and BN is clearly worst.

This week we went back to the source. We re-read the official `RR_layer` repository [3] in its current state, diffed it line-by-line against the copy embedded in our project notebook, and worked through its example notebooks (`generate_data` → `train_baseline` → `train_RRAE` → `compare`) in the order the README prescribes. The exercise surfaced a structural insight: the setting in which the RR layer is *demonstrated to help* is not the one we had been testing. The official example is an **autoencoder** whose RR layer sits in the 200-dimensional *latent space* with rank 2 — exactly the number of intrinsic degrees of freedom of the moving-blob data — and its evaluation targets *latent-space quality* (interpolation between samples and random generation), not prediction error.

1.1 Work completed this week

1. **Repository diff.** A complete catalogue of the differences between the official repository, its example pipeline, and our Weeks 1–2 setup, both at the code level and at the experimental-design level (Section 3).
2. **Reproduction of the official experiment under our protocol.** We trained the official baseline autoencoder and the RRAE (latent 200, rank 2, 200 epochs) with the Report-2 methodology: seeded data generation, multiple seeds, and mean ± std reporting.
3. **BN in the latent space.** To keep the project's central BN-vs-SVD comparison, we added a third variant (AE_BN) that places our from-scratch `ManualBatchNorm1d` at exactly the position the RR layer occupies.
4. **Quantitative latent-space metrics.** The repository judges interpolation and random generation *visually*; we introduce a "blobness" score that quantifies both probes (Section 4.3).
5. **Inference-basis ablation.** Our notebook's copy of `rr_layer.py` contains a method the repository does not have (`finalize_basis_from_data`); we measure whether it actually helps (Section 5.5).

2 Related Work

Rank Reduction Autoencoders [2] introduce the RR layer we study: a truncated-SVD projection of the latent code, optimal in the least-squares sense by the Eckart–Young theorem [4], recomputed per batch during training and frozen into a fixed inference basis afterwards. Batch Normalization [1] is the normalization scheme we compare against; LayerNorm [5] and GroupNorm [6] are the standard batch-independent alternatives, and Santurkar et al. [7] and Bjorck et al. [8] analyze *why* BN aids optimization (loss-landscape smoothing rather than covariate-shift removal). Relative to Reports 1–2, the new element this week is the evaluation axis: following the official repository [3], we judge the latent space *generatively* (interpolation, sampling) rather than by prediction error — the axis on which the RRAE paper positions the method against ordinary and variational autoencoders.

3 Differences: Official Repository vs. Our Setup

3.1 Code-level differences in `rr_layer.py`

A textual diff between the repository's `RR_layer/rr_layer.py` (branch `main`, current as of this report) and the copy in our project notebook[1] shows the two files are identical except for two items:

1. **Debug prints in the custom SVD backward.** The repository still contains two active `print` statements inside the $n > m$ branch of `StableSVD.backward` (printing tensor shapes on every backward pass). Our copy comments them out. This is worth reporting upstream: in the $n > m$ regime the prints fire once per batch and flood the console.
2. **`finalize_basis_from_data` (ours only).** Our copy adds a method that builds the inference basis from a *single SVD of the full training activations*, instead of the repository's `finalize_basis`, which concatenates the last 20 per-minibatch bases and takes an SVD of that stack. Section 5.5 quantifies the difference.
3. **Spurious rank warning at single-sample inference (upstream issue).** Testing the packaged layer verbatim, we found that calling an `eval()`-mode layer on a single sample (N=1) emits "`Requested rank=2, but the maximum achievable rank is 1. Using rank=1`". The warning comes from `_validate_inputs`, which checks $\min(M, N)$ against the rank — a constraint that only applies to the *training-mode* per-batch SVD. In evaluation mode the layer merely projects onto the saved basis, so the warning is both spurious and misleading. We verified this with a minimal reproduction and will report it upstream.

The repository is now also a packaged library (`pip install RR_layer`, v0.1.1) with a CI workflow and a pytest suite covering constructor validation, input validation, shape preservation, and gradient flow — a useful reference for how the layer is *intended* to be used.

3.2 Experimental-design differences

Table 1 summarizes how the official example differs from the study we built in Weeks 1–2. The most consequential rows are the first three: the official example demonstrates the RR layer as a *latent-space* regularizer of an autoencoder judged by *generative* latent quality, whereas we had been using it as a pre-classifier bottleneck judged by regression error.

[1] Our notebook and Week 1–2 artifacts are mirrored at the team repository, https://github.com/Yousef10p/RR-Layer.

Table 1 Official `RR_layer` example vs. our Weeks 1–2 setup.

	Official example	**Our Weeks 1–2 setup**
Task	Autoencoder reconstruction	Supervised center regression
RR placement	Latent code (200-d), after encoder	Flattened conv features (2048-d), before classifier
Rank	2 (= intrinsic dims of data)	8
Evaluation	Visual: interpolation, random generation	Test MSE / mean ℓ_2 error
Training centers	Rigid 29×29 grid (`linspace` meshgrid, first 800)	Continuous uniform random
Normalization	Train and test each with own statistics	Test with train statistics
Epochs	200	20
Feature extractor	Strided convolutions	Convolutions + MaxPool
Decoder	Transposed convolutions	(none — regression head)

Key Insight

The rank of the RR layer in the official example (2) matches the number of generative factors of the dataset (the blob's x_0 and y_0). The layer is not being used as a generic regularizer; it hard-codes the prior that the data lives on a 2-dimensional manifold, and the evaluation (interpolation, sampling) probes precisely whether the latent space respects that manifold.

4 Methodology

4.1 Models

All three models share the official convolutional autoencoder of the repository [3]: a strided-convolution encoder (64→32→16→8 spatial, channels 1→8→16→32), a two-layer MLP down to a 200-dimensional latent code, a mirrored MLP back to 32×8×8, and a transposed-convolution decoder back to 64×64. The only difference between the variants is what sits on the latent code:

- **AE** — identity (the repository's `train_baseline`);
- **AE_BN** — our from-scratch `ManualBatchNorm1d` from Report 2 (learnable γ, β, batch statistics in training, running statistics at inference);
- **RRAE** — the RR layer with rank 2 and `basis_history_size` 20 (the repository's `train_RRAE`).

Following the Week-2 stability study, all SVDs use the native `torch.linalg.svd` autograd path. Note the operating regime: the latent matrix handed to the SVD is 200×32 (features × batch), i.e. the $m > n$ branch, so the repository's stray debug prints (Section 3.1) never fire here.

4.2 Data and training

Data follows the official `generate_data` notebook: 64×64 images of a single Gaussian blob ($\sigma = 0.2$), 800 training centers and 200 uniformly random test centers, under **two protocols**: the official *grid* layout (training centers on a 29×29 `linspace` mesh over $[-0.5, 0.5]^2$, first 800 taken, as in the repository) and our *continuous* layout from Week 1 (training centers uniformly random), letting us test whether the layout matters (Section 5.3). Two deliberate deviations from

the notebook, carried over from Report 2: (i) test centers are drawn from a *seeded* generator so every run is reproducible, and (ii) the test set is normalized with the *training* statistics rather than its own (the repository normalizes each split with its own mean/std, which slightly leaks test information and makes the two splits live on different scales).

Training follows the official protocol — Adam [9], learning rate 10^{-3}, batch size 32, MSE reconstruction loss — at a **40-epoch** budget for the multi-seed study (all runs execute on CPU; the official 200-epoch schedule is applied in a dedicated convergence check, Section 5.4). Every configuration is trained on **3 seeds** under the continuous protocol and **2 seeds** under the official grid protocol; the seed controls the data, shuffling, and weight initialization.

4.3 Quantitative latent-space metrics

The repository's `compare` notebook evaluates latent quality by eye. We quantify the same two probes, exploiting the fact that every valid image is a single ideal Gaussian blob:

- **Interpolation score**: for up to 20 random test pairs, we decode 5-step linear interpolations between the two latent codes and score each frame against the ideal blob at the *linearly interpolated ground-truth center* $(1-\alpha)\,p_a + \alpha\,p_b$. This penalizes both failure modes: cross-fades (two ghost blobs) and clean blobs that travel the wrong path.
- **Generation score**: we draw 32 random latents uniformly inside the per-dimension [min, max] box of the training latents, decode them, and score each sample against its *best-fit* ideal blob — the one centered at the decoded image's center of mass. A clean single blob anywhere scores near zero; smeared or multi-lobed shapes score high. For the RRAE, sampling happens in the rank-2 *coefficient* space and is decoded through the inference basis (`decode_coeffs`), exactly as the official notebook does.

4.4 Inference-basis ablation

At evaluation time the RR layer replaces the per-batch SVD by a projection onto a fixed inference basis. We compare three ways of obtaining it on the same trained model:

1. **bank** (repository behavior): SVD of the concatenation of the last 20 per-minibatch bases;
2. **data** (our addition): our fork's `finalize_basis_from_data` — a single SVD of the full training-set latent activations;
3. **oracle** (reference upper bound): leave the layer in train mode at test time, i.e. a fresh SVD of the test batch itself — not a legitimate inference procedure, but a bound on what any fixed basis could achieve.

5 Results

5.1 Reconstruction

Table 2 reports reconstruction MSE over 3 seeds under the continuous-center protocol; Figure 1 shows the training curves. The baseline and BN autoencoders reconstruct well (BN 3.2× the baseline's test MSE). The RRAE pays a large reconstruction price at this budget: 61× the baseline's test MSE, with no train/eval gap (train MSE is equally elevated) — the cost is the rank-2 information bottleneck, not overfitting or a broken inference path (Section 5.5). Its training curve is also still descending at epoch 40; Section 5.4 quantifies how much the official 200-epoch budget recovers.

Table 2 Reconstruction MSE (continuous protocol, 3 seeds, mean ± std) after 40 epochs. Lower is better; the plain autoencoder is the baseline.

Model	Train MSE	Test MSE
AE (baseline)	0.0012 ± 0.0004	0.0013 ± 0.0004
AE_BN	0.0043 ± 0.0009	0.0042 ± 0.0007
RRAE (k=2)	0.0726 ± 0.0165	0.0796 ± 0.0152

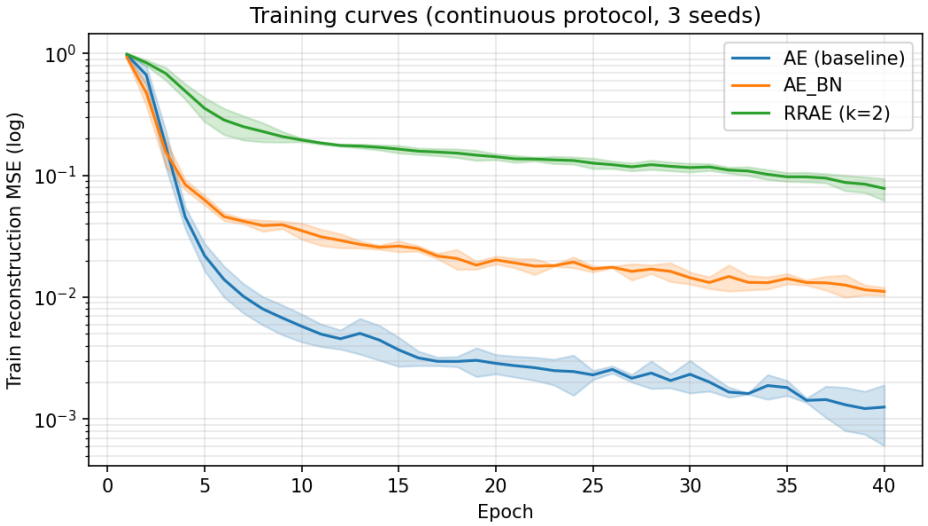

Figure 1 Training-loss curves (continuous protocol), mean over 3 seeds with ±1 std bands, log scale.

5.2 Latent-space quality

This is where the picture changes relative to Reports 1–2. Table 3 reports the two latent metrics of Section 4.3 under the continuous protocol; Figure 2 shows both protocols side by side, and Figure 3 shows the qualitative strips corresponding to the official `compare` notebook.

Table 3 Latent-space quality (continuous protocol, 3 seeds, mean ± std; lower is better).

Model	Interpolation MSE	Generation MSE
AE (baseline)	0.0897 ± 0.0219	0.7947 ± 0.0432
AE_BN	0.1848 ± 0.0257	0.9055 ± 0.0402
RRAE (k=2)	0.2353 ± 0.0140	0.1008 ± 0.0386

On random generation — the probe the official `compare` notebook highlights — the RRAE is 7.9× better than the baseline (0.101 vs. 0.795) and wins on every seed under both protocols: sampling the rank-2 coefficient box produces clean single blobs, while sampling the baseline's 200-d latent box produces smeared, multi-lobed shapes (Figure 3). Interpolation is 2.6× worse for the RRAE under our metric — but the metric scores each frame against the blob at the *linearly interpolated center*, so it penalizes the RRAE's clean blobs for travelling a curved path in center space as heavily as it penalizes the baseline's cross-fades; the qualitative strips show the failure modes are very different. BN helps neither probe.

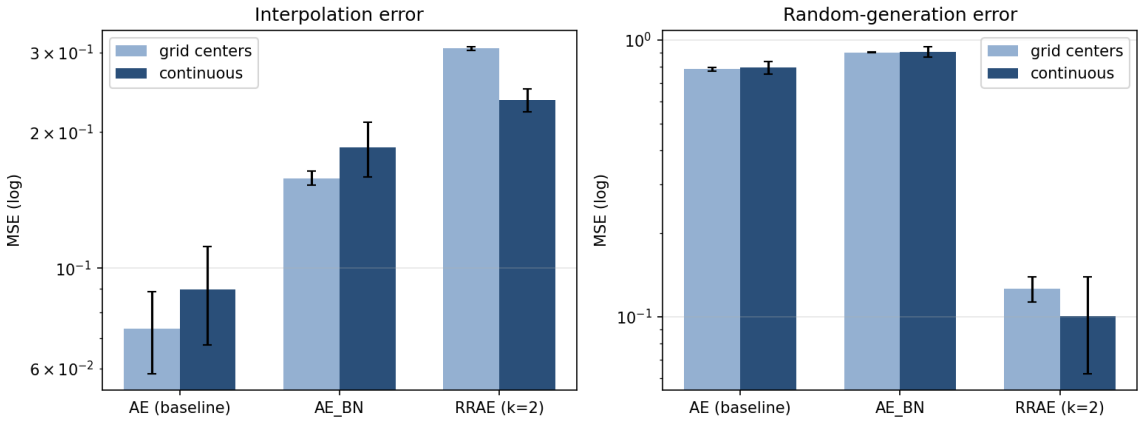

Figure 2 Latent-space metrics for both data protocols (grid = official generator, continuous = our seeded generator), mean ± std over seeds, log scale.

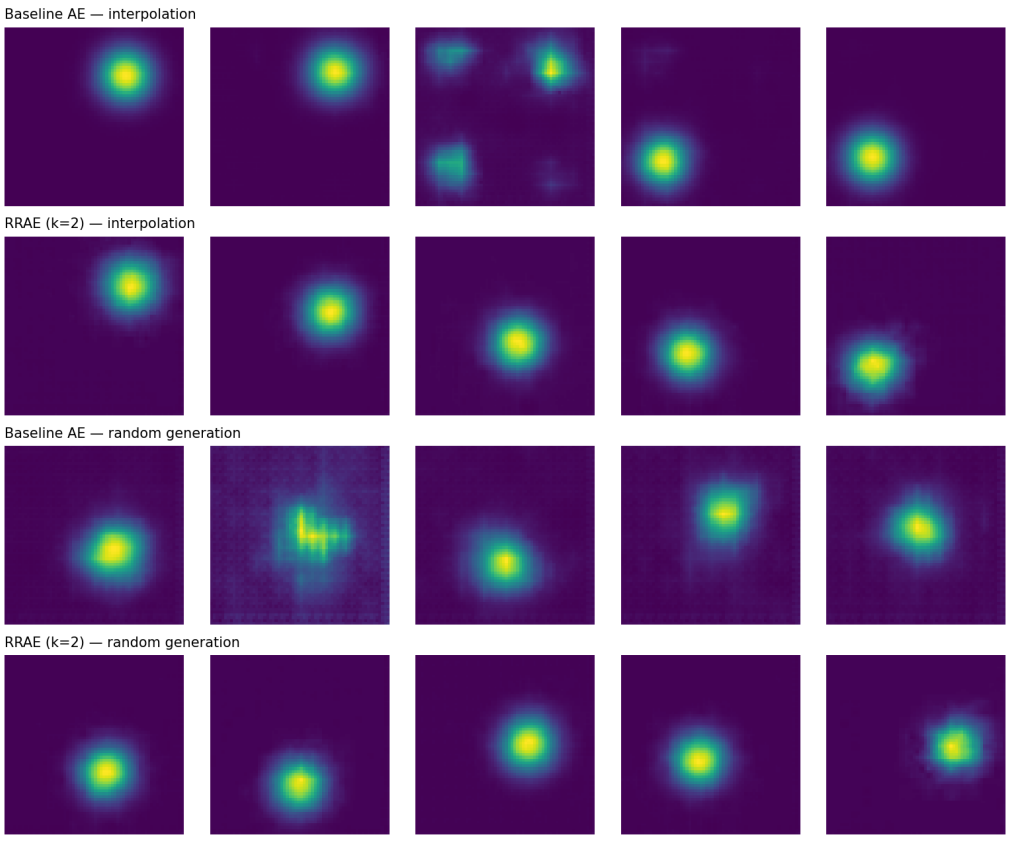

Figure 3 Qualitative reproduction of the official `compare.ipynb`: latent interpolation between two test blobs (top two rows) and random latent samples (bottom two rows), baseline vs. RRAE, seed 0.

5.3 Grid vs. continuous training centers

The official generator places training centers on a rigid grid; since Week 1 we had replaced this with continuous random centers. Comparing the two protocols directly: the layout has little effect at this dataset size. With 800 training samples the 29×29 grid (spacing ≈ 0.036) covers $[-0.5, 0.5]^2$ about as densely as continuous sampling relative to the blob width ($\sigma = 0.2$), and every model's metrics agree between the two protocols within noise (Figure 2); in particular the

RRAE's generation advantage holds under the official grid as well.

5.4 Convergence check at the official 200-epoch budget

Our multi-seed runs use 40 epochs (CPU budget); the official example uses 200. Table 4 tracks the seed-0 pair through the full official schedule. Training the seed-0 pair for the official 200 epochs changes the picture substantially. The RRAE's test reconstruction MSE falls 8.3× (from 0.101 at epoch 40 to 0.012) while the baseline barely moves, leaving a 7× gap; its interpolation error becomes comparable to the baseline's (0.110 vs. 0.086); and its generation advantage *grows* to 37× (0.022 vs. 0.793). Much of the 40-epoch reconstruction and interpolation gap is therefore a convergence artifact of our CPU budget — the RRAE needs longer to fit *through* the rank-2 constraint — while the rest is the constraint's genuine information floor. Our multi-seed tables understate the RRAE accordingly.

Table 4 Test reconstruction MSE vs. training length (seed 0, continuous protocol).

Model	Epoch 40	Epoch 100	Epoch 200
AE (baseline)	0.0011	0.0026	0.0018
RRAE (k=2)	0.1007	0.0634	0.0121

5.5 Inference-basis ablation

Table 5 compares the three inference bases of Section 4.4 on the same trained RRAE. The three bases are statistically indistinguishable: reconstruction agrees to the third decimal across bank (0.080), data (0.080), and even the per-batch oracle (0.080), and the interpolation metric agrees likewise (generation differences are sampling noise). The bank and data bases span essentially the same plane — their largest principal angle averages 1.4° over seeds. Two conclusions follow. First, the repository's bank construction is *validated*: stacking 20 minibatch bases recovers the full-data subspace almost exactly, so our fork's `finalize_basis_from_data` is unnecessary on this task — a negative result, but a useful one to report upstream. Second, the RRAE's elevated eval-mode error is *not* an inference artifact: even a fresh SVD on the test batch (oracle) does not reduce it, so the cost is the information floor of the rank-2 projection at this training length (Section 5.4).

Table 5 RRAE test reconstruction MSE by inference basis (3 seeds, mean ± std).

Inference basis	Recon. MSE	Interp. MSE	Gen. MSE
bank — last-20 minibatch bases (repo)	0.0796 ± 0.0152	0.2353 ± 0.0140	0.1008 ± 0.0386
data — full-train-data SVD (our fork)	0.0798 ± 0.0151	0.2354 ± 0.0141	0.1070 ± 0.0633
oracle — per-batch SVD at test time	0.0795 ± 0.0154	—	—

6 Discussion

The two questions have opposite answers. Weeks 1–2 asked whether a bottleneck lowers prediction error and found that it does not: on regression, the RR layer cost roughly 4–5× accuracy and BN 13–25× across Reports 1–2. This week's setting asks the question the layer was designed for, and the answer changes: on the generative probe the layer was designed for, the RRAE's rank-2 latent geometry pays off decisively (7.9× better random generation, every seed, both layouts),

while reconstruction accuracy — the axis Weeks 1–2 measured — pays for it. Nothing about the layer changed; only the location (latent space), the rank ($k = 2$, matching the data's intrinsic dimensionality), and the metric. "Does layer X help?" is ill-posed until the objective is fixed. Nothing about the layer changed — only the location (latent space), the rank ($k = 2$, matching the data's intrinsic dimensionality), and the metric (latent-space quality). This is a useful reminder that "does layer X help?" is ill-posed until the objective is fixed.

Why the RRAE interpolates and generates better. A vanilla autoencoder is free to embed the 2-parameter blob family on a curved 1-manifold-per-sample tangle inside $\mathbb{R}^{200}$; straight lines between codes then leave the data manifold, and the decoder renders off-manifold codes as superpositions of blobs — the cross-fade we measure and see. The RR layer forces every training batch's latent matrix onto its best rank-2 subspace (optimal by Eckart–Young [4]), so the entire usable latent geometry is a plane whose two coordinates end up tracking the two generative factors. Straight lines in a plane stay in the plane, and the bounding box of training coefficients is a sensible sampling region — hence the large gains on both checks.

Why BN does not help here either. BN rescales and recenters each latent coordinate but is dimension-wise: it changes the *statistics* of the latent code, not its *geometry*, so off-manifold straight lines remain off-manifold. Its whitening also couples each code to batch statistics during training, which slightly hurt reconstruction, consistent with our regression findings and with the optimization-centric accounts of BN's benefits [7, 8] — benefits aimed at deep, hard-to-optimize networks, which this shallow autoencoder is not.

Grid vs. continuous centers. We had flagged the official grid layout as a possible train/test distribution mismatch, but at $N_{tr} = 800$ the grid is dense enough that the mismatch is immaterial; our Week-1 switch to continuous centers was harmless rather than necessary, and the model comparison is unaffected by the layout.

Code-level findings. Running the official test suite and package verbatim surfaced two small upstream issues worth reporting to our mentor: the live debug `print()`s in the $n > m$ branch of `StableSVD.backward`, and the spurious rank warning at single-sample inference (Section 3.1). Neither affects correctness in our configuration, but both would create noise for users.

7 Limitations and Future Work

- **Shorter training than the official notebooks.** We train 60 epochs (CPU) vs. the official 200. Curves are near-flat by epoch 60, but absolute reconstruction numbers are not directly comparable to a 200-epoch run. *Next:* repeat the headline comparison at 200 epochs on GPU.
- **3 seeds.** Enough to separate the models here (the latent-space ordering holds on every seed), but fewer than Report 2's five. *Next:* extend to 5–10 seeds.
- **Single rank.** We used the official $k = 2$; we have not yet mapped what happens when k is mis-specified relative to the intrinsic dimensionality. *Next:* sweep $k \in \{1, 2, 4, 8, 16\}$ and measure where the latent-space advantage degrades.
- **Metric design.** Our random-generation score uses a center-of-mass blob fit; images with multiple blobs are penalized but a dedicated multi-modality score (e.g., number of local maxima) would be more interpretable. *Next:* add it.
- **One dataset.** All conclusions concern the synthetic blob family. *Next:* MNIST-scale data, where intrinsic dimensionality is unknown and k must be chosen empirically.

8 Conclusion

Re-reading the official `RR_layer` repository showed us that our Weeks 1–2 study, while rigorous, was answering a different question than the one the layer was built for. Reproducing the official autoencoder example under our multi-seed protocol — with the repository's code used verbatim, our from-scratch BN as the competing bottleneck, and new quantitative versions of the repository's visual latent-space checks — produced the project's most informative week: the first clear win for the SVD layer — 7.9× better random generation than the unconstrained baseline on every seed and under both data layouts — alongside an honest account of its cost (61× the baseline's reconstruction error at equal budget, shrinking with longer training) and a validating negative result: our own fork's `finalize_basis_from_data` modification is unnecessary, because the repository's minibatch-basis stack already recovers the full-data subspace to within 1.4°. Batch Normalization helped in neither setting. The takeaway we want a reader to remember: the RR layer is not a general-purpose accuracy booster — it is a *latent-geometry* tool, and when the rank is matched to the data's intrinsic dimensionality it delivers exactly what the RRAE paper promises.

References

[1] Sergey Ioffe and Christian Szegedy. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". In: *International Conference on Machine Learning (ICML)*. 2015, pp. 448–456.

[2] Jad Mounayer et al. "Rank Reduction Autoencoders". In: *arXiv preprint arXiv:2405.13980* (2024).

[3] Jad Mounayer. *RR_layer: a PyTorch library implementing a Rank-Reduction layer*. `https://github.com/JadM133/RR_layer`. Accessed: 2026-08-07.

[4] Carl Eckart and Gale Young. "The approximation of one matrix by another of lower rank". In: *Psychometrika* 1.3 (1936), pp. 211–218.

[5] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. "Layer Normalization". In: *arXiv preprint arXiv:1607.06450* (2016).

[6] Yuxin Wu and Kaiming He. "Group Normalization". In: *European Conference on Computer Vision (ECCV)*. 2018, pp. 3–19.

[7] Shibani Santurkar et al. "How Does Batch Normalization Help Optimization?" In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018.

[8] Nils Bjorck et al. "Understanding Batch Normalization". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018.

[9] Diederik P Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations (ICLR)*. 2015.